

GURUGRAM POLICE

CYBER SECURITY SUMMER INTERNSHIP

(GPCSSI) 2025



SkidScan- Dark web Monitor & Web Forensics

TEAM MEMBERS: -

Vibhu Yadav (CW-405)
Dushyant Nagal (CW-111)
Chirag Solanki (CW-84)
Prem Singh (CW-169)
Janvi Sorout (CW-147)

MENTOR: -

Dr. Rakshit Tandon



Acknowledgment

*We would like to express our sincere gratitude to the **Gurugram Police Cyber Security Cell** for providing us with the opportunity to participate in the **GPCSSI 2025** summer internship program. It has been an incredibly enriching experience that has helped us grow both technically and professionally.*

*A special thank you to **Dr. Rakshit Tandon**, whose mentorship, guidance, and deep insight into the field of cyber security inspired us throughout the development of **SkidScan** — a tool designed to aid in **dark web monitoring** and **web forensics**. His encouragement and constructive feedback played a crucial role in shaping the direction and impact of this project.*

*We are also thankful to the entire team behind **GPCSSI '25** for fostering a collaborative and learning-rich environment. This internship not only enhanced our technical skills but also gave us a glimpse into the real-world applications of cyber security in law enforcement.*

*Developing **SkidScan** has been a rewarding journey, and we are truly grateful for the opportunity to contribute to a cause as vital as digital safety and cybercrime investigation.*



Abstract

***SkidScan** is a Python-based cyber investigation tool developed during the Gurugram Police Cyber Security Summer Internship (GPCSSI 2025), aimed at supporting real-time monitoring and digital forensics for law enforcement agencies. The tool is composed of two primary modules: **Black Mirror**, a dark web crawler, and **ChronoLeak Hunter**, a web forensics engine.*

*The **Black Mirror** module enables investigators to monitor and extract content from .onion domains by leveraging Tor routing, enabling anonymous crawling of dark web spaces where illicit activity often occurs. It integrates intelligent filtering and keyword-based content extraction to flag suspicious material relevant to ongoing cybercrime investigations.*

*The **ChronoLeak Hunter** module utilizes **Wayback Machine archives** to analyse the historical footprint of websites. It implements a machine learning pipeline using **TF-IDF vectorization** and **K-Means clustering** to identify anomalous or malicious changes in web content over time. This assists cyber professionals in uncovering data leaks, tampering, or previously hosted malicious content on websites.*

*Built entirely with Python and key libraries like **Scikit-learn**, **TextBlob**, **NumPy**, and **Pandas**, SkidScan offers a lightweight yet powerful solution for cyber cops to enhance their visibility into dark web threats and conduct structured forensic investigations.*



Introduction

In today's digital landscape, where cyber threats continue to evolve in both complexity and reach, law enforcement and cybercrime units require efficient tools to monitor hidden platforms and investigate digital footprints. **SkidScan** is a Python-based cybersecurity tool developed as part of the **Gurugram Police Cyber Security Summer Internship (GPCSSI 2025)** to empower cyber cops with real-time dark web monitoring and automated web forensics capabilities. SkidScan is designed with two core modules:

- **Black Mirror** – a dark web crawler that anonymously explores and extracts data from onion domains.
- **ChronoLeak Hunter** – a forensic analysis engine that utilizes archived web data to identify potential leaks or malicious activity through machine learning.

The goal of SkidScan is to provide cyber investigators with a streamlined solution that supports both **proactive monitoring** and **post-incident analysis**, reducing manual effort and improving the speed and accuracy of cybercrime investigations.

Technology Stack: -

SkidScan is developed entirely in **Python**, making it lightweight, flexible, and easily extensible. The following libraries and tools form the backbone of the tool:

- ✓ **Requests & URL parse** – For handling HTTP requests and parsing URLs
- ✓ **TextBlob** – For basic sentiment analysis and textual preprocessing
- ✓ **Scikit-learn** – For implementing machine learning models (TF-IDF vectorization, K-Means clustering)
- ✓ **NumPy & Pandas** – For data handling and analysis
- ✓ **Stem (Tor Controller Library)** – To signal and rotate Tor circuits while crawling onion sites anonymously
- ✓ **Wayback Machine API** – To fetch and analyze archived versions of websites for digital forensics



Module 1: Black Mirror

(Dark Web Reconnaissance & Keyword Monitoring)

Overview: -

*Black Mirror is the first module of the SkidScan suite, designed to perform **dark web reconnaissance** by interacting with .onion websites hosted on the Tor network. It enables cybercrime analysts to identify which dark web links are live and actively serving content, followed by crawling those sites to extract sensitive intelligence like credentials, financial terms, or administrative access references.*

*This module plays a critical role in **passive data collection and early-stage threat intelligence**, especially for tracking illegal marketplaces, hidden forums, and compromised data dumps. By narrowing down live .onion services and matching on high-risk keywords, investigators can prioritize which sources need deeper inspection.*

Key Features: -

- ✓ **Live Onion Service Detection:** Verifies reachability of .onion sites through Tor routing.
- ✓ **Keyword-based Content Scanning:** Identifies the presence of sensitive terms like "login", "password", "wallet", etc.
- ✓ **Modular Design:** Built as two cooperating scripts (check_alive.py and run.py) for flexibility.
- ✓ **Anonymity Preserved:** All traffic is routed via the Tor network to maintain operational security.
- ✓ **Structured Output:** Results are stored in clean, readable text files for further analysis or correlation.



Technology Used: -

- ✓ **Language:** Python 3.x
 - ✓ **Networking:** requests (via Tor SOCKS proxy)
 - ✓ **Proxy Configuration:** socks5h://127.0.0.1:9050 (default Tor local proxy)
 - ✓ **File I/O:** For input/output URL management (config/sites.txt, exports/results.txt)
 - ✓ **Execution Environment:** Optimized for Kali Linux, VMs, or sandboxed systems with the Tor service running
-

Working: -

Step 1: check_alive.py

- **Purpose:** Determine which .onion URLs from config/sites.txt are currently live.
- **Method:**
 - ✓ Loads list of .onion sites.
 - ✓ Sends HTTP requests through the local Tor proxy.
 - ✓ Filters out unreachable sites or those returning non-200 status codes.
- **Output:**
config/sites_alive.txt → Contains verified live .onion services.
- **Step 2: run.py**
- **Purpose:** Crawl live .onion pages and identify sensitive terms in the HTML content.
- **Method:**
 - ✓ Reads .onion URLs (from sites.txt or sites_alive.txt).
 - ✓ Routes all connections via Tor proxy.



- ✓ Searches each page's content for keywords like:
"login", "password", "email", "bitcoin", "wallet", "admin",
"secure".
- ✓ Stores match in a structured format.

➤ **Output:**

exports/results.txt → Each line: URL → [matched keywords].



Module 2: ChronoLeak Hunter

(Web Forensics & Historical Analysis with AI)

Overview: -

ChronoLeak Hunter is the web forensics engine of the SkidScan suite. This module leverages the **Wayback Machine (Internet Archive)** to retrieve, analyse, and assess archived versions of web content for potential data leaks, misconfigurations, and sensitive directory disclosures.

It automates the identification of high-risk directories and files by combining **pattern matching, content intelligence, and AI-powered path clustering and prediction**. The module enables cybercrime units and digital forensic teams to reconstruct a domain's historical exposure — uncovering traces of leaked credentials, exposed configuration files, and development directories that may have since been removed but are still accessible via archive snapshots.

Key Features: -

- **Historical Path Extraction** via Wayback Machine
- **AI-enhanced Risk Scoring** for URLs and paths
- **Sensitive Directory Detection** (e.g., /admin, /backup, /.git)
- **AI Content Analysis** for exposed credentials, tech stacks, and interesting files
- **Path Clustering** using **TF-IDF Vectorization** and **K-Means Clustering**
- **Smart Path Prioritization** based on structural and semantic features
- **Detection of Protected Directories** (403/401) and misconfigurations

Technologies Used: -

- **Python Libraries:** requests, textblob, scikit-learn, NumPy, pandas, concurrent.futures, urllib, stem
- **AI Components:**
 - ✓ **TF-IDF Vectorizer:** Converts URL paths into numerical feature vectors.
 - ✓ **K-Means Clustering:** Groups structurally and semantically similar paths.
 - ✓ **Path Risk Predictor:** Uses pattern-matching, depth, length, and known suspicious indicators to score URLs.
 - ✓ **TextBlob:** Analyses textual content for sentiment and language clues.



Working: -

ChronoLeak Hunter operates by first accepting a single domain, a list of domains, or an auto-discovery mode with optional date range filters. It then queries the Wayback Machine to retrieve archived URLs, including subdomains, and extracts unique paths. These paths are clustered using TF-IDF vectorization and K-Means clustering, and each path is scored based on sensitivity indicators and structure. The top 500 high-risk paths are scanned live to identify directory listings, file disclosures, and access-restricted areas. For each accessible directory, AI-driven content analysis detects credentials, tech stacks, emails, and sensitive files such as .env or config.php. Results are logged with ai_risk_score, detailed ai_content_analysis, and stored in JSON, TXT, or CSV format for further investigation.

Available input options (CLI Only): -

🔧 Command Line Options - Explained	
Option	Description
<code>-d</code> , <code>--domain</code>	Specifies a single target domain to analyze.
<code>-f</code> , <code>--file</code>	Provides a file containing multiple domains (one per line).
<code>-auto</code>	Enables AI auto-discovery mode to scan the domain and its subdomains.
<code>-t</code> , <code>--threads</code>	Sets the number of concurrent threads to use (default is 10).
<code>--delay</code>	Adds a delay (in seconds) between requests to avoid rate-limiting.
<code>-o</code> , <code>--output</code>	Defines the file name to save output results.
<code>--format</code>	Chooses the output format: <code>json</code> , <code>csv</code> , or <code>txt</code> (default is <code>json</code>).
<code>-v</code> , <code>--verbose</code>	Enables verbose mode for detailed logs and debug messages.
<code>--no-ai</code>	Disables all AI-enhanced features (runs in basic mode).
<code>--date-from</code>	Limits analysis to snapshots taken <i>after</i> this date (<code>YYYYMMDD</code>).
<code>--date-to</code>	Limits analysis to snapshots taken <i>before</i> this date (<code>YYYYMMDD</code>).



Sample Output Elements: -

- ✓ ***ai_risk_score***: A value between 0.0 and 1.0 indicating the severity of the finding.
- ✓ ***ai_content_analysis***: A dictionary showing:
 - ***contains_credentials***: True/False
 - ***technology_stack***: [PHP, WordPress]
 - ***interesting_files***: [.env, backup.sql]
- ✓ ***is_directory_listing***: Boolean flag for directory indexing



Screenshots: -

